

Exercise 5.1: Temperature Data (DWD)

Analysis of temperature measurements from the Deutscher Wetterdienst (DWD) across 80 stations in Germany (1781–2024).

In [3]: `%pip install seaborn -qqq`

Note: you may need to restart the kernel to use updated packages.

```
In [4]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns
from scipy import stats

plt.rcParams['figure.dpi'] = 120
plt.rcParams['font.size'] = 11
sns.set_theme(style='whitegrid', palette='muted')

MONTH_NAMES = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
               'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']
```

Task 1 – Import & Clean Data

```
In [5]: # ----- Load -----
df = pd.read_csv('temperature_data_processed.csv',
                 dtype={'stationid': int, 'date': str, 'time': int, 'temp': float})

# ----- parse date -----
df['date'] = pd.to_datetime(df['date'], format='%Y%m%d')
df['year'] = df['date'].dt.year
df['month'] = df['date'].dt.month
df['day'] = df['date'].dt.day

# ----- remove missing values -----
before = len(df)
df = df[df['temp'] != -999].copy()
after = len(df)

print(f"Rows before cleaning : {before:,}")
print(f"Rows removed (-999) : {before - after:,}")
print(f"Rows after cleaning : {after:,}")
df.head()
```

```
Rows before cleaning : 6,972,481
Rows removed (-999) : 15,812
Rows after cleaning : 6,956,669
```

Out[5]:

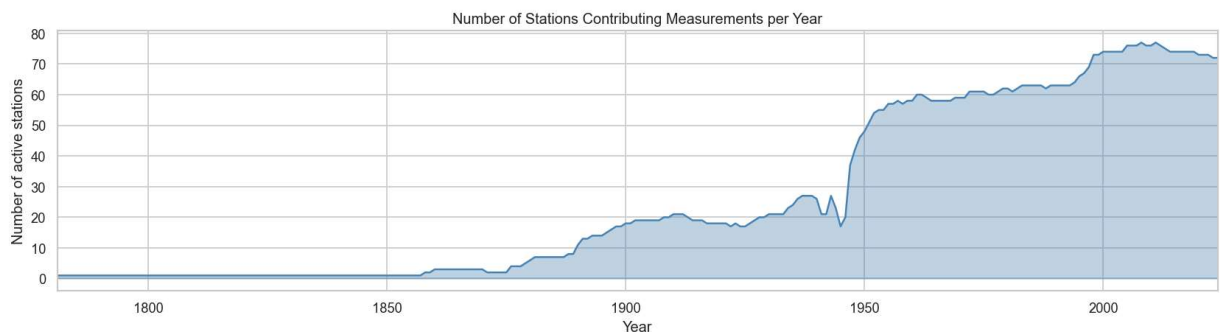
	stationid	temp	date	time	year	month	day
0	3	-5.6	1891-01-01	7	1891	1	1
1	3	-0.7	1891-01-01	14	1891	1	1
2	3	-4.0	1891-01-01	21	1891	1	1
3	3	-5.2	1891-01-02	7	1891	1	2
4	3	-1.0	1891-01-02	14	1891	1	2

Task 2 – Active Stations per Year

```
In [6]: stations_per_year = (
    df.groupby('year')['stationid']
      .nunique()
      .rename('n_stations')
      .reset_index()
    )

fig, ax = plt.subplots(figsize=(14, 4))
ax.fill_between(stations_per_year['year'], stations_per_year['n_stations'],
               alpha=0.35, color='steelblue')
ax.plot(stations_per_year['year'], stations_per_year['n_stations'],
        color='steelblue', linewidth=1.4)
ax.set_xlabel('Year')
ax.set_ylabel('Number of active stations')
ax.set_title('Number of Stations Contributing Measurements per Year')
ax.set_xlim(stations_per_year['year'].min(), stations_per_year['year'].max())
ax.yaxis.set_major_locator(mticker.MaxNLocator(integer=True))
plt.tight_layout()
plt.savefig('stations_per_year.png', bbox_inches='tight')
plt.show()

print(stations_per_year.describe())
```



	year	n_stations
count	238.00000	238.000000
mean	1905.12605	26.579832
std	69.45199	27.808718
min	1781.00000	1.000000
25%	1846.25000	1.000000
50%	1905.50000	18.000000
75%	1964.75000	58.000000
max	2024.00000	77.000000

Task 3 – Reference Stations (active in both 1960 and 2020)

```
In [7]: stations_1960 = set(df[df['year'] == 1960]['stationid'].unique())
stations_2020 = set(df[df['year'] == 2020]['stationid'].unique())

reference_stations = stations_1960 & stations_2020

print(f"Stations active in 1960      : {len(stations_1960)}")
print(f"Stations active in 2020      : {len(stations_2020)}")
print(f"Reference stations (intersection) : {len(reference_stations)}")
print(f"Station IDs: {sorted(reference_stations)}")
```

```
Stations active in 1960      : 58
Stations active in 2020      : 73
Reference stations (intersection) : 53
Station IDs: [np.int64(164), np.int64(183), np.int64(232), np.int64(282), np.int64(403), np.int64(433), np.int64(691), np.int64(722), np.int64(880), np.int64(891), np.int64(1048), np.int64(1270), np.int64(1346), np.int64(1358), np.int64(1420), np.int64(1468), np.int64(1612), np.int64(1639), np.int64(1684), np.int64(1975), np.int64(2014), np.int64(2115), np.int64(2261), np.int64(2290), np.int64(2483), np.int64(2559), np.int64(2667), np.int64(2812), np.int64(3015), np.int64(3032), np.int64(3126), np.int64(3196), np.int64(3631), np.int64(3668), np.int64(3730), np.int64(3761), np.int64(3987), np.int64(4104), np.int64(4177), np.int64(4271), np.int64(4336), np.int64(4466), np.int64(4625), np.int64(4887), np.int64(4911), np.int64(4931), np.int64(5100), np.int64(5371), np.int64(5397), np.int64(5516), np.int64(5705), np.int64(5792), np.int64(5906)]
```

Task 4 – Filter Data

```
In [8]: mask = (
    df['stationid'].isin(reference_stations) &
    df['year'].between(1960, 2020) &
    df['time'].isin([12, 14])
)

df_filtered = df[mask].copy()

print(f"Rows in filtered dataset: {len(df_filtered):,}")
print(f"Unique stations      : {df_filtered['stationid'].nunique()}")
```

```
print(f"Year range : {df_filtered['year'].min()} - {df_filtered['year'].max()}")
df_filtered.head()
```

```
Rows in filtered dataset: 1,160,070
Unique stations : 53
Year range : 1960 - 2020
```

```
Out[8]:
```

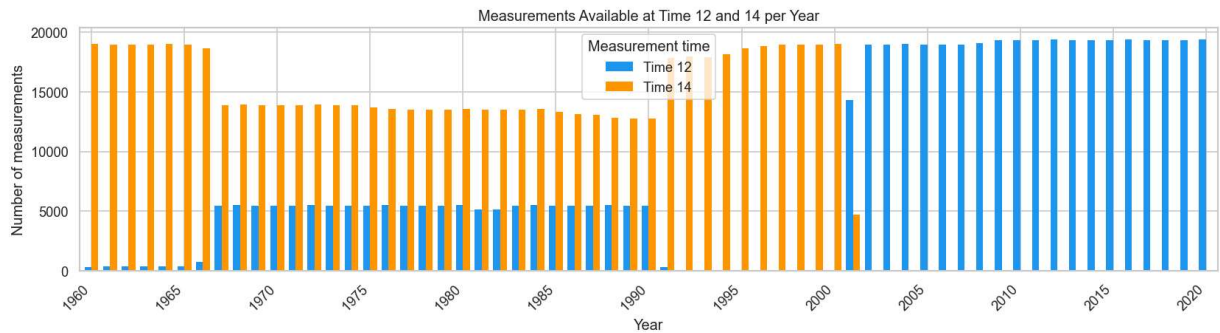
	stationid	temp	date	time	year	month	day
197597	164	6.6	1960-01-01	14	1960	1	1
197600	164	8.4	1960-01-02	14	1960	1	2
197603	164	5.3	1960-01-03	14	1960	1	3
197606	164	5.0	1960-01-04	14	1960	1	4
197609	164	5.9	1960-01-05	14	1960	1	5

Task 5 – Measurements at Time 12 vs 14 per Year

```
In [9]: counts_by_time = (
    df_filtered.groupby(['year', 'time'])
        .size()
        .unstack(level='time')
        .rename(columns={12: 'Time 12', 14: 'Time 14'})
        .fillna(0)
        .astype(int)
)

fig, ax = plt.subplots(figsize=(14, 4))
counts_by_time.plot(kind='bar', ax=ax, width=0.75,
                    color=['#2196F3', '#FF9800'], edgecolor='none')
ax.set_xlabel('Year')
ax.set_ylabel('Number of measurements')
ax.set_title('Measurements Available at Time 12 and 14 per Year')
ax.legend(title='Measurement time')
# show every 5th x-tick label to avoid crowding
ticks = ax.get_xticks()
labels = [l.get_text() for l in ax.get_xticklabels()]
ax.set_xticks(ticks[::5])
ax.set_xticklabels(labels[::5], rotation=45, ha='right')
plt.tight_layout()
plt.savefig('measurements_by_time.png', bbox_inches='tight')
plt.show()

print(counts_by_time.head(10).to_string())
```



time	Time 12	Time 14
year		
1960	329	19032
1961	365	18980
1962	365	18980
1963	365	18980
1964	366	19032
1965	365	18980
1966	730	18615
1967	5475	13870
1968	5490	13908
1969	5475	13870

Task 6 – Monthly Temperature Trends

6a – Violin plot (distribution per month)

```
In [10]: fig, ax = plt.subplots(figsize=(14, 5))

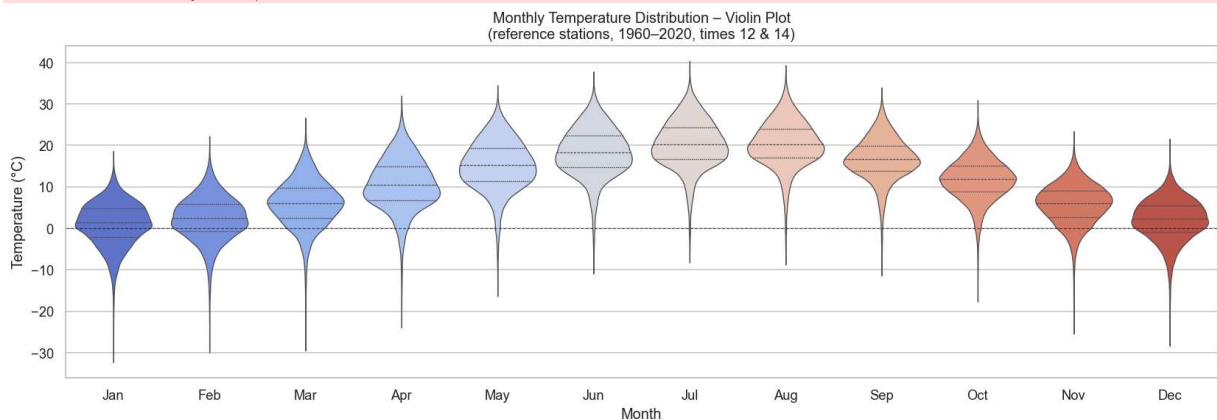
sns.violinplot(
    data=df_filtered,
    x='month', y='temp',
    palette='coolwarm',
    inner='quartile',
    linewidth=0.8,
    ax=ax
)

ax.set_xticks(range(12))
ax.set_xticklabels(MONTH_NAMES)
ax.set_xlabel('Month')
ax.set_ylabel('Temperature (°C)')
ax.set_title('Monthly Temperature Distribution - Violin Plot\n(reference stations,
ax.axhline(0, color='black', linewidth=0.7, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.savefig('monthly_violin.png', bbox_inches='tight')
plt.show()
```

C:\Users\Umer\AppData\Local\Temp\ipykernel_11020\2229939758.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.violinplot(
```

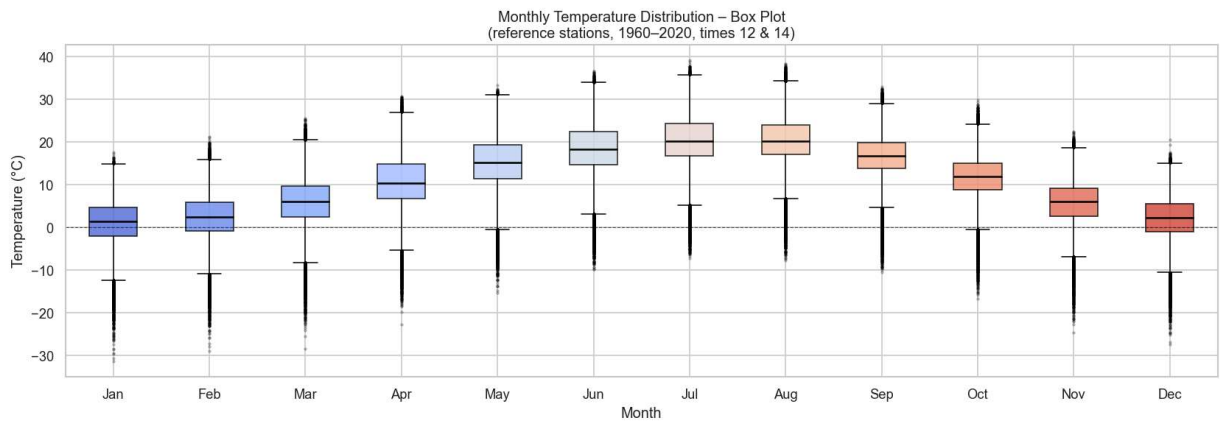


6b – Box plot

```
In [11]: fig, ax = plt.subplots(figsize=(14, 5))

month_colors = sns.color_palette('coolwarm', 12)
bp = ax.boxplot(
    [df_filtered[df_filtered['month'] == m]['temp'].values for m in range(1, 13)],
    patch_artist=True,
    medianprops=dict(color='black', linewidth=1.5),
    whiskerprops=dict(linewidth=0.9),
    capprops=dict(linewidth=0.9),
    flierprops=dict(marker='o', markersize=1.5, alpha=0.3, linestyle='none'),
    showfliers=True
)
for patch, color in zip(bp['boxes'], month_colors):
    patch.set_facecolor(color)
    patch.set_alpha(0.8)

ax.set_xticks(range(1, 13))
ax.set_xticklabels(MONTH_NAMES)
ax.set_xlabel('Month')
ax.set_ylabel('Temperature (°C)')
ax.set_title('Monthly Temperature Distribution - Box Plot\n(reference stations, 1960-2020, times 12 & 14)')
ax.axhline(0, color='black', linewidth=0.7, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.savefig('monthly_boxplot.png', bbox_inches='tight')
plt.show()
```



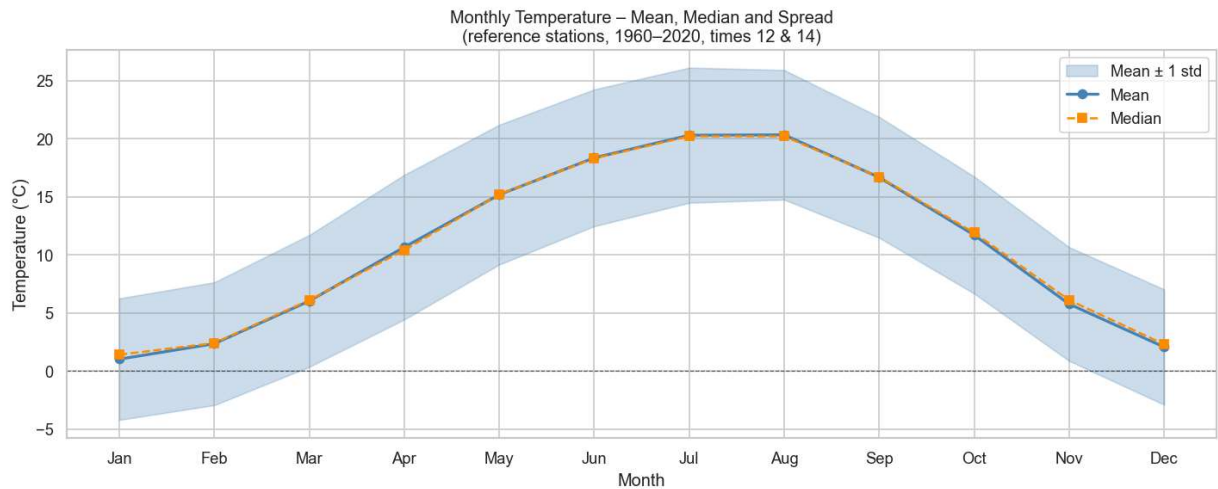
6c – Mean $\pm$ 1 std ribbon with jittered strip

```
In [12]: monthly_stats = (
    df_filtered.groupby('month')['temp']
        .agg(['mean', 'std', 'median'])
        .reset_index()
)

fig, ax = plt.subplots(figsize=(12, 5))

ax.fill_between(
    monthly_stats['month'],
    monthly_stats['mean'] - monthly_stats['std'],
    monthly_stats['mean'] + monthly_stats['std'],
    alpha=0.25, color='steelblue', label='Mean  $\pm$  1 std'
)
ax.plot(monthly_stats['month'], monthly_stats['mean'],
        'o-', color='steelblue', linewidth=2, label='Mean', zorder=3)
ax.plot(monthly_stats['month'], monthly_stats['median'],
        's--', color='darkorange', linewidth=1.5, label='Median', zorder=3)

ax.set_xticks(range(1, 13))
ax.set_xticklabels(MONTH_NAMES)
ax.set_xlabel('Month')
ax.set_ylabel('Temperature (°C)')
ax.set_title('Monthly Temperature – Mean, Median and Spread\n(reference stations, 1960–2020, times 12 & 14)')
ax.axhline(0, color='black', linewidth=0.7, linestyle='--', alpha=0.6)
ax.legend()
plt.tight_layout()
plt.savefig('monthly_ribbon.png', bbox_inches='tight')
plt.show()
```



Task 7 – Long-Term Temperature Trend (1960–2020)

7a – All months combined: yearly mean with linear trend

```
In [13]: yearly_mean = (
    df_filtered.groupby('year')['temp']
        .agg(mean='mean', std='std', median='median',
             q25=lambdax: x.quantile(0.25),
             q75=lambdax: x.quantile(0.75))
        .reset_index()
    )

# OLS linear regression for trend line
slope, intercept, r, p, se = stats.linregress(yearly_mean['year'], yearly_mean['mean'])
trend_y = slope * yearly_mean['year'] + intercept

fig, ax = plt.subplots(figsize=(14, 5))

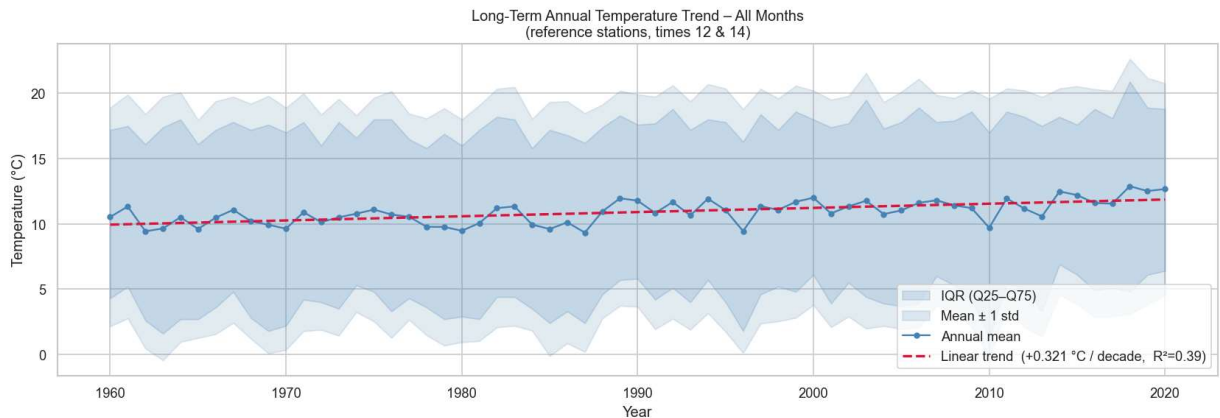
ax.fill_between(yearly_mean['year'],
               yearly_mean['q25'], yearly_mean['q75'],
               alpha=0.20, color='steelblue', label='IQR (Q25-Q75)')
ax.fill_between(yearly_mean['year'],
               yearly_mean['mean'] - yearly_mean['std'],
               yearly_mean['mean'] + yearly_mean['std'],
               alpha=0.15, color='steelblue', label='Mean ± 1 std')
ax.plot(yearly_mean['year'], yearly_mean['mean'],
        'o-', color='steelblue', markersize=4, linewidth=1.4, label='Annual mean')
ax.plot(yearly_mean['year'], trend_y,
        color='crimson', linewidth=2, linestyle='--',
        label=f'Linear trend ({slope*10:+.3f} °C / decade, R²={r**2:.2f})')

ax.set_xlabel('Year')
ax.set_ylabel('Temperature (°C)')
ax.set_title('Long-Term Annual Temperature Trend – All Months\n(reference stations, 1960–2020)')
ax.legend()
plt.tight_layout()
```



```
plt.savefig('trend_all_months.png', bbox_inches='tight')
plt.show()

print(f"Linear trend: {slope*10:+.4f} °C per decade    (p = {p:.4f})")
```



Linear trend: +0.3206 °C per decade (p = 0.0000)

7b – Selected month (June = 6): yearly mean with linear trend

```
In [14]: SELECTED_MONTH = 6 # change as desired

df_month = df_filtered[df_filtered['month'] == SELECTED_MONTH]

yearly_mean_month = (
    df_month.groupby('year')['temp']
        .agg(mean='mean', std='std',
             q25=lambda x: x.quantile(0.25),
             q75=lambda x: x.quantile(0.75))
        .reset_index()
)

slope_m, intercept_m, r_m, p_m, _ = stats.linregress(
    yearly_mean_month['year'], yearly_mean_month['mean']
)
trend_y_m = slope_m * yearly_mean_month['year'] + intercept_m

fig, ax = plt.subplots(figsize=(14, 5))

ax.fill_between(yearly_mean_month['year'],
               yearly_mean_month['q25'], yearly_mean_month['q75'],
               alpha=0.20, color='darkorange', label='IQR (Q25-Q75)')
ax.fill_between(yearly_mean_month['year'],
               yearly_mean_month['mean'] - yearly_mean_month['std'],
               yearly_mean_month['mean'] + yearly_mean_month['std'],
               alpha=0.15, color='darkorange', label='Mean ± 1 std')
ax.plot(yearly_mean_month['year'], yearly_mean_month['mean'],
        'o-', color='darkorange', markersize=4, linewidth=1.4,
        label=f'{MONTH_NAMES[SELECTED_MONTH-1]} annual mean')
ax.plot(yearly_mean_month['year'], trend_y_m,
        color='crimson', linewidth=2, linestyle='--',
        label=f'Linear trend ({slope_m*10:+.3f} °C / decade, R²={r_m**2:.2f})')

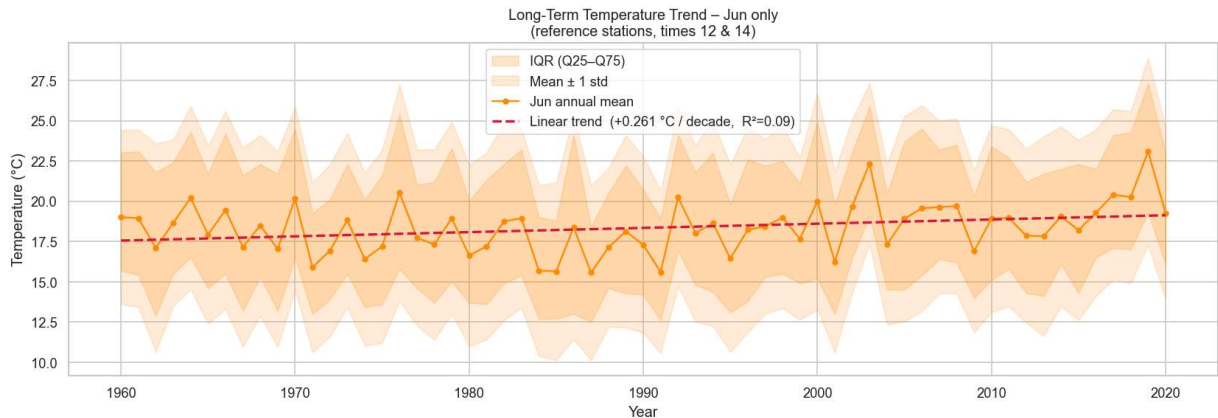
ax.set_xlabel('Year')
```

```

ax.set_ylabel('Temperature (°C)')
ax.set_title(f'Long-Term Temperature Trend - {MONTH_NAMES[SELECTED_MONTH-1]} only\n'
            f'(reference stations, times 12 & 14)')
ax.legend()
plt.tight_layout()
plt.savefig(f'trend_month_{SELECTED_MONTH}.png', bbox_inches='tight')
plt.show()

print(f"Linear trend ({MONTH_NAMES[SELECTED_MONTH-1]}): {slope_m*10:+.4f} °C per de

```



Linear trend (Jun): +0.2614 °C per decade (p = 0.0200)

7c – Heatmap: all 12 months over all years (split-apply-combine)

```

In [15]: pivot = (
    df_filtered.groupby(['year', 'month'])['temp']
        .mean()
        .unstack('month')
    )
pivot.columns = MONTH_NAMES

fig, ax = plt.subplots(figsize=(14, 10))
sns.heatmap(
    pivot,
    cmap='RdBu_r',
    center=0,
    linewidths=0,
    yticklabels=5,
    cbar_kws={'label': 'Mean temperature (°C)', 'shrink': 0.6},
    ax=ax
)
ax.set_xlabel('Month')
ax.set_ylabel('Year')
ax.set_title('Mean Monthly Temperature Heatmap (1960-2020)\n'
            '(reference stations, times 12 & 14)')
plt.tight_layout()
plt.savefig('heatmap_monthly_yearly.png', bbox_inches='tight')
plt.show()

```

